

# Leveraging Grammar and Reinforcement Learning for Neural Program Synthesis

Шапка Павел (МФТИ)

Московский Физико-Технический Институт

21 апреля 2026 г.

# Постановка задачи

**Домен:** мир Karel

Задача: по набору вход-выходных примеров

$$\mathcal{D} = \{(\{\text{IO}_i^k\}_{k=1\dots K}, \lambda_i)\} \quad \lambda_i(I_i^k) = O_i^k$$

выучить сеть  $\sigma$ , такую, что по заданным входам-выходам она генерирует программу:

$$\sigma : \{\text{IO}^k\}_{k=1\dots K} \rightarrow \hat{\lambda},$$

которая бы при исполнении роботом Karel переводила вход в выход  
 $\hat{\lambda}(I) = O$

# Теоретическая справка

Всего есть два режима обучения: **supervised** и **beam\_rl**

- 1 В **supervised** режиме лосс задается просто функцией кросс-энтропии

$$\mathcal{L}_{\text{MLE}} = - \sum_{i=1}^N \sum_{t=1}^L \log p_{\theta}(s_t^i \mid s_{<t}^i, \text{IO}_i),$$

где  $L$  - длина программы

- 2 В **supervised** режиме функция награды задается аналогом формулы в REINFORCE

$$\mathcal{L}_{\text{R}}(\theta) = \sum_{i=1}^N \sum_{\lambda \in \text{BS}(p_{\theta}, S)} q_{\theta}(\lambda \mid \text{IO}_i) \cdot R_i(\lambda),$$

где  $\text{BS}(p_{\theta}, S)$  - результат beam search,  $q_{\theta}$  - распределение, построенное на отобранных примерах (перенормированное распределение на beam'ах),  $R_i$  - функция награды

Также присутствуют проверки на корректность генерируемых программ:

- 1 **use grammar** - статический чеккер, маска перед softmax, соответствующие недопустимым токенам в  $-\infty$ , обнуляя их вероятность генерации
- 2 **learn grammar** - LSTM модел, корректирующая распределение на токенах  $\sigma$ . Модель учится истинные токены, которые точно есть в программах не трогать (прибавлять 0), а к остальным токенам давать прибавку, ведущую к уменьшению их вероятностей

Также за токена паддинга программа не штрафует (потому что они не влияют на работу)

# Постановка эксперимента

**Цель:** Обучить модель в режиме supervised, далее дофайнтюнить с помощью beam RL

Репозиторий: GitHub

Сбор данных: загрузка датасета, далее разбиение его на необходимые части с помощью python-функции

Запуск с помощью функций, приведенных в README.md оригинального проекта

Режимы обучения: supervised, RL Beam

# Результаты

| Model                  | Data | EM-1 | Generalize-1 | Generalize-5 | Generalize-50 |
|------------------------|------|------|--------------|--------------|---------------|
| MLE + grammar          | 1%   | 11.1 | 15.5         | 25.7         | 39.4          |
| MLE + grammar          | 3%   | X    | X            | X            | X             |
| MLE + grammar          | 10%  | X    | X            | X            | X             |
| beam_rl + grammar      | 1%   | 23.6 | 35.4         | X            | X             |
| beam_rl + grammar      | 3%   | X    | X            | X            | X             |
| beam_rl + grammar      | 10%  | X    | X            | X            | X             |
| MLE + learn_syntax     | 1%   | 9.1  | 12.2         | 21.6         | 35.4          |
| MLE + learn_syntax     | 3%   | X    | X            | X            | X             |
| MLE + learn_syntax     | 10%  | X    | X            | X            | X             |
| beam_rl + learn_syntax | 1%   | 9.0  | 12.2         | 15.9         | 22.5          |
| beam_rl + learn_syntax | 3%   | X    | X            | X            | X             |
| beam_rl + learn_syntax | 10%  | X    | X            | X            | X             |

**Таблица:** Results on the Karel dataset across different training data fractions. EM = Exact Match, Generalize = Full Generalization on held-out IO pairs.